

Problem Set Solutions: Seminar 11

Search Trees (1)
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: Search trees, AVL trees, augmentation, split/merge, order statistics
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Search trees are the first data structure in the course where the same small interface becomes a universal engine: ordered set operations, predecessor/successor search, interval counting, online medians, range sums, gcd maintenance, and order statistics are all obtained by preserving a logarithmic-height tree and augmenting each node by a small summary of its subtree. The intellectual point is not only that balanced trees are fast, but that their recursive structure makes local metadata globally useful.

Contents

Problem 1. Checking whether a rooted binary tree is a search tree	2
Problem 2. Minimum, maximum, and successor in a search tree	2
Problem 3. Closed form for Fibonacci numbers	3
Problem 4. Independent sets in a cycle	4
Problem 5. Stopping rules for AVL insertion	5
Problem 6. Stopping rules for AVL deletion	6
Problem 7. Joining two AVL trees	7
Problem 8. Dynamic median under insertions and deletions	7
Problem 9. Counting intersecting pairs of intervals from two dynamic families	8
Problem 10. Set with range-sum queries and global add	9
Problem 11. Maintaining the gcd of a dynamic set	10
Problem 12. Maintaining order statistics in a dynamic set	10

Conventions. All keys are assumed distinct unless explicitly stated otherwise. A balanced binary search tree may be implemented as an AVL tree, a red-black tree, or a randomized treap; in this sheet all stated bounds are worst-case for AVL and expected/amortized for treaps when split/merge is used. For AVL trees write

$$\Delta(v) = h(\text{right}(v)) - h(\text{left}(v)).$$

Thus an AVL vertex satisfies $|\Delta(v)| \leq 1$.

Problem 1. Checking whether a rooted binary tree is a search tree

Given a rooted binary tree, check in linear time whether it is a binary search tree.

Solution.

Idea. The defining property of a BST is global, not merely local: every key in the left subtree of v must be smaller than $\text{key}(v)$, and every key in the right subtree must be larger. This can be checked by carrying allowed intervals, or equivalently by verifying that the inorder traversal is strictly increasing.

Algorithm. Perform an inorder traversal: visit the left subtree, then the vertex itself, then the right subtree. Maintain the previously seen key $prev$. Whenever a new key x is visited, check that $prev < x$. If the check fails, the tree is not a BST. Otherwise set $prev = x$ and continue.

Equivalently, a recursive procedure may return three values for each subtree: whether it is a BST, its minimum key, and its maximum key. A vertex v is valid iff both children are valid and

$$\max(\text{left}(v)) < \text{key}(v) < \min(\text{right}(v)),$$

with empty subtrees ignored.

Correctness. In every BST, the inorder traversal visits keys in sorted increasing order: all keys of the left subtree are smaller than the root, and all keys of the right subtree are larger. Applying this recursively gives a strictly increasing sequence.

Conversely, suppose the inorder sequence is strictly increasing. Consider any vertex v . All vertices in its left subtree are visited before v , so their keys are smaller than $\text{key}(v)$. All vertices in its right subtree are visited after v , so their keys are larger than $\text{key}(v)$. Therefore the BST condition holds at every vertex, hence the whole tree is a search tree.

Running time. Each vertex is visited once and only $O(1)$ work is done per vertex. The time is $O(n)$. The recursion stack is $O(h)$, where h is the tree height; with an explicit stack the same bound holds.

Problem 2. Minimum, maximum, and successor in a search tree

How do we find the minimum and maximum element of a BST? How do we find the smallest element larger than x ?

Solution.

Answer. The minimum is obtained by repeatedly going to the left child; the maximum is obtained by repeatedly going to the right child. The successor of x can be found during one descent from the root.

Algorithm. For the minimum:

```
node* minimum(node* v) {
    while (v->left != nullptr) v = v->left;
    return v;
}
```

For the maximum, replace `left` by `right`.

To find the smallest key strictly larger than x , keep a candidate answer:

```
node* upper_bound(node* root, Key x) {
    node* ans = nullptr;
    node* v = root;
    while (v != nullptr) {
        if (v->key > x) {
            ans = v;
            v = v->left;
        } else {
            v = v->right;
        }
    }
    return ans;
}
```

If we are given a pointer to the node containing x and each node stores a parent pointer, the successor can also be found as follows. If x has a right subtree, the successor is the minimum of that right subtree. Otherwise climb by parent pointers until we move from a left child to its parent; that parent is the successor.

Correctness. The minimum cannot be in a right subtree while a left child exists, because every key in the left subtree is smaller than the current key. Therefore repeatedly going left reaches the unique minimum. The maximum is symmetric.

For `upper_bound`, every time $\text{key}(v) > x$, the current vertex is a valid candidate, but a smaller valid candidate may still lie in its left subtree, so we go left. Every time $\text{key}(v) \leq x$, neither v nor any vertex in its left subtree can be the answer, so we go right. Thus no possible answer is discarded, and the last stored candidate is exactly the smallest key greater than x .

Running time. All procedures follow one root-to-leaf path, hence take $O(h)$ time, where h is the height of the tree. In a balanced tree this is $O(\log n)$.

Problem 3. Closed form for Fibonacci numbers

Let $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. Let

$$\varphi = \frac{1 + \sqrt{5}}{2}.$$

Prove that

$$F_n = \frac{\varphi^n - (-\varphi)^{-n}}{\sqrt{5}} \quad (n \geq 0).$$

Solution.

Idea. This is Binet's formula. The second root of the characteristic equation $t^2 = t + 1$ is

$$\psi = \frac{1 - \sqrt{5}}{2} = -\frac{1}{\varphi} = (-\varphi)^{-1}.$$

So the claimed formula is

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

Correctness. First check the base cases:

$$\frac{\varphi^0 - \psi^0}{\sqrt{5}} = 0 = F_0,$$

while

$$\frac{\varphi - \psi}{\sqrt{5}} = \frac{\frac{1+\sqrt{5}}{2} - \frac{1-\sqrt{5}}{2}}{\sqrt{5}} = 1 = F_1.$$

Both φ and ψ satisfy $t^2 = t + 1$, so for every $n \geq 2$,

$$\varphi^n = \varphi^{n-1} + \varphi^{n-2}, \quad \psi^n = \psi^{n-1} + \psi^{n-2}.$$

Subtracting and dividing by $\sqrt{5}$ gives

$$\frac{\varphi^n - \psi^n}{\sqrt{5}} = \frac{\varphi^{n-1} - \psi^{n-1}}{\sqrt{5}} + \frac{\varphi^{n-2} - \psi^{n-2}}{\sqrt{5}}.$$

Thus the right-hand side satisfies the same recurrence and the same initial values as F_n . By induction it equals F_n for all $n \geq 0$.

Running time. This is a proof, not an algorithm.

Problem 4. Independent sets in a cycle

Let C_n be the cycle graph on $n \geq 3$ vertices. Find the number of independent sets in C_n .

Solution.

Idea. First solve the path. For a path P_n , an independent set either excludes the first vertex or includes it and therefore excludes the second vertex.

Answer. The number of independent sets in C_n is

$$F_{n-1} + F_{n+1}.$$

This number is also the n -th Lucas number.

Correctness. Let p_n be the number of independent sets in the path P_n . Then

$$p_n = p_{n-1} + p_{n-2},$$

because either the first vertex is not chosen, leaving an independent set of P_{n-1} , or it is chosen, forcing the second vertex not to be chosen and leaving an independent set of P_{n-2} . The initial values are $p_0 = 1$ and $p_1 = 2$, hence

$$p_n = F_{n+2}.$$

Now fix a vertex v in C_n . If v is not chosen, then the remaining graph is a path on $n-1$ vertices, contributing

$$p_{n-1} = F_{n+1}$$

independent sets. If v is chosen, then its two neighbors are forbidden, and the remaining graph is a path on $n-3$ vertices, contributing

$$p_{n-3} = F_{n-1}$$

independent sets. These two cases are disjoint and exhaustive. Therefore the answer is $F_{n+1} + F_{n-1}$.

Running time. The formula is closed form once Fibonacci numbers are known. Computing it iteratively takes $O(n)$ arithmetic operations, or $O(\log n)$ with matrix exponentiation/fast doubling.

Problem 5. Stopping rules for AVL insertion

Prove the standard stopping rules for insertion in an AVL tree: after updating a vertex v , if $\Delta(v) = 0$ we may stop; if $\Delta(v) = \pm 1$, then $h(v)$ increased; and one rotation is enough.

Solution.

Idea. After inserting a leaf, only vertices on the search path from the inserted leaf to the root can change height. At each such vertex, exactly one child subtree may have increased its height by one. The balance factor tells us whether this local increase propagates upward.

Correctness. Consider a vertex v on the upward path. Before insertion, $|\Delta(v)| \leq 1$. Only one child height can increase, and it increases by at most one.

There are three cases.

Case 1. The new balance factor is 0. Then before the insertion the two child heights differed by one, and the shorter child has just caught up. Hence the height of v is unchanged. Since ancestors can change only if the height of their child changes, propagation stops.

Case 2. The new balance factor is ± 1 . Then before the insertion the two child heights were equal, and one of them has increased by one. Therefore

$$h(v) = 1 + \max(h(\text{left}(v)), h(\text{right}(v)))$$

increases by one. This increase may affect the parent, so the algorithm must continue upward.

Case 3. The new balance factor is ± 2 . Then insertion occurred in the heavier child of a vertex that was already leaning in that direction. A single AVL rebalancing step is performed: either one simple rotation (LL or RR case) or one double rotation (LR or RL case). In both subcases, a direct height calculation shows that the height of the rebalanced subtree becomes equal to the height of the same subtree before insertion. Therefore no ancestor height changes, so the algorithm stops after this one rebalancing step.

Thus the stated stopping rules are correct, and insertion needs at most one AVL rebalancing step.

Running time. Searching for the insertion position takes $O(h)$ time. Recomputing balances on the upward path takes $O(h)$ time. Since an AVL tree has $h = O(\log n)$, insertion takes $O(\log n)$ time and performs at most one local rebalancing step.

Problem 6. Stopping rules for AVL deletion

Prove the standard stopping rules for deletion in an AVL tree: after updating a vertex v , if $\Delta(v) = \pm 1$ we may stop; if $\Delta(v) = 0$, then $h(v)$ decreased. Is one rotation always enough?

Solution.

Idea. Deletion is the mirror image of insertion only until the first rotation. A child subtree may become shorter. Sometimes this height decrease stops; sometimes it propagates through many ancestors. Therefore deletion may require many rebalancing steps.

Correctness. After deleting a node, only vertices on the path from the physical deletion point to the root may change height. At each such vertex exactly one child height may have decreased by one.

Again consider a vertex v after its child height has been updated.

- Case 1.** If the new balance factor is ± 1 , then before deletion the two child heights were equal. One child became shorter, so the maximum child height remains the old value. Hence $h(v)$ does not decrease. Since the parent sees no height change, the upward process stops.
- Case 2.** If the new balance factor is 0, then before deletion the two child heights differed by one, and the taller child has just decreased. Now both child heights are equal and the maximum child height has decreased by one. Therefore $h(v)$ decreases by one, so the deletion effect propagates to the parent.
- Case 3.** If the new balance factor is ± 2 , the vertex must be rebalanced by a single or double AVL rotation. Unlike insertion, after this rotation the height of the whole rebalanced subtree may decrease by one. If it decreases, the parent must also be processed. Thus rotations can propagate upward.

Therefore the stopping rules are valid: stop when the new balance factor is ± 1 ; continue when the subtree height decreases.

Answer. No, one rotation is not always enough. There are AVL trees in which deleting a leaf from the shorter side of a subtree makes that subtree shorter after rebalancing; the same situation repeats at the parent, then at its parent, and so on. In particular, minimal AVL trees of consecutive heights exhibit this behavior: a deletion near the shallow side can force rebalancing at $\Theta(\log n)$ ancestors.

Running time. Deletion takes $O(\log n)$ time in an AVL tree. In the worst case it may perform $\Theta(\log n)$ rotations, although each individual rotation is local.

Problem 7. Joining two AVL trees

Given two AVL trees T_1 and T_2 such that every key of T_1 is smaller than every key of T_2 , build an AVL tree containing their union in time $O(\log(|T_1| + |T_2|))$.

Solution.

Idea. Use a separator key and attach the shorter tree at the correct height inside the taller tree. This is the AVL *join* operation.

Algorithm. Assume first that $h(T_1) \leq h(T_2)$; the other case is symmetric. Remove the maximum key x from T_1 , producing T'_1 . Then all keys in T'_1 are smaller than x , and x is smaller than all keys in T_2 .

Walk down the left spine of T_2 until reaching the first subtree U whose height is at most $h(T'_1) + 1$. Let p be the parent pointer at which this subtree was attached. Create a new node z with key x , left child T'_1 , and right child U . Replace U by z in T_2 . Now rebalance the vertices on the path from p back to the root.

If $h(T_1) > h(T_2)$, remove the minimum key from T_2 and perform the mirror operation along the right spine of T_1 .

Correctness. The BST order is preserved because

$$\text{all keys of } T'_1 < x < \text{all keys of } T_2.$$

When z is inserted, its two child heights differ by at most one by construction, so z itself is AVL-balanced. All subtrees not on the modified spine remain unchanged and valid. The only possible AVL violations are on the path back to the root, and ordinary AVL rebalancing restores the invariant there without changing the inorder order. Hence the final tree is an AVL tree whose key set is exactly the union of the two original key sets.

Running time. Deleting the maximum/minimum from an AVL tree costs $O(\log n)$. The descent follows at most $|h(T_2) - h(T_1)| + O(1)$ edges, and the rebalancing path has the same order. Since AVL height is $O(\log n)$, the total time is $O(\log(|T_1| + |T_2|))$.

Problem 8. Dynamic median under insertions and deletions

Elements are inserted into and deleted from an initially empty set. After each query, report the median of the current set in $O(\log n)$ time.

Solution.

Idea. Maintain the lower half and upper half in two balanced search trees. The median is the boundary between the halves.

Algorithm. Maintain two balanced BSTs L and R with the invariants

$$|L| \in \{|R|, |R| + 1\}, \quad \max L \leq \min R$$

whenever both trees are nonempty. Thus L stores the lower half and R stores the upper half. We report the lower median $\max L$; if the convention requires the upper median, swap the size convention.

Insertion: compare x with $\max L$ if L is nonempty. Insert x into L if $x \leq \max L$, otherwise into R . Then rebalance by moving $\max L$ to R or $\min R$ to L until the size invariant holds.

Deletion: delete x from whichever tree contains it, then rebalance in the same way.

Correctness. The order invariant implies every element of L is at most every element of R . The size invariant implies that L contains exactly the lower half of the current set, including the middle element when the size is odd. Therefore $\max L$ is exactly the lower median. Rebalancing moves only boundary elements, so it preserves the order invariant and restores the size invariant after every update.

Running time. Each insertion/deletion in a balanced BST costs $O(\log n)$. Rebalancing moves at most one element between the two trees, also in $O(\log n)$ time. Reporting the median is $O(1)$ if each tree stores pointers to its minimum/maximum, or $O(\log n)$ without them.

Problem 9. Counting intersecting pairs of intervals from two dynamic families

There are two sets of intervals on a line. Intervals are only added. After each addition, report how many pairs consisting of one interval from the first set and one interval from the second set intersect.

Solution.

Idea. It is easier to count new intersections by subtracting new non-intersections. A closed interval $[l, r]$ is disjoint from $[c, d]$ iff $d < l$ or $c > r$.

Algorithm. For each family maintain two order-statistic trees: one containing all left endpoints and one containing all right endpoints. Also maintain the current answer Ans .

When adding an interval $[l, r]$ to the first family, let the second family contain m intervals. The number of second-family intervals disjoint from $[l, r]$ is

$$\#\{[c, d] : d < l\} + \#\{[c, d] : c > r\}.$$

Both quantities are rank queries in the endpoint trees. Hence the number of new intersecting pairs is

$$m - \#\{d < l\} - \#\{c > r\}.$$

Add this to Ans , then insert l and r into the endpoint trees of the first family. Addition to the second family is symmetric.

Correctness. Every pair counted before the update remains present because intervals are never deleted. Thus only pairs involving the newly added interval can change the answer. For a fixed new interval $[l, r]$, an old interval $[c, d]$ fails to intersect it exactly in the two mutually exclusive cases $d < l$ and $c > r$. Therefore subtracting these two counts from the size of the opposite family gives exactly the number of new intersecting pairs. Adding that number to the previous answer preserves the invariant that Ans equals the total number of intersecting cross-family pairs.

Running time. Each insertion and each rank query in an order-statistic tree costs $O(\log n)$ for the corresponding family size. Thus each update is processed in $O(\log n + \log m)$ time.

Problem 10. Set with range-sum queries and global add

Maintain a set S under: add x , delete x , find the sum of elements in $[l, r]$, and add x to every element of S . Answer every query in $O(\log q)$ time, except global add in $O(1)$.

Solution.

Idea. A global shift should not move every key explicitly. Store all keys after subtracting a global offset. Then global addition changes only the offset. Range sums are handled by split/merge and subtree sums.

Algorithm. Maintain a variable add , initially 0. Instead of storing the actual value x , store the normalized value

$$y = x - add.$$

Use a balanced BST or treap keyed by y . Each node stores its subtree size and the sum of normalized keys in its subtree.

- **addValue(x):** insert $x - add$ into the tree.
- **eraseValue(x):** erase $x - add$ from the tree.
- **addAll(t):** set $add \leftarrow add + t$.
- **rangeSum(l, r):** normalized keys that correspond to actual values in $[l, r]$ lie in $[l - add, r - add]$. Split the tree into three parts: keys $< l - add$, keys in $[l - add, r - add]$, and keys $> r - add$. If the middle part has normalized sum S and size c , the actual sum is

$$S + c \cdot add.$$

Then merge the three parts back.

Correctness. At all times, for every stored element,

$$\text{actual value} = \text{normalized key} + \text{add}.$$

Insertion and deletion use exactly this relation, so they affect the intended actual value. A global addition by t increases every actual value by t when we replace add by $\text{add} + t$, without changing normalized keys. A range $[l, r]$ in actual coordinates is therefore exactly $[l - \text{add}, r - \text{add}]$ in normalized coordinates. The augmented subtree sum gives the sum of normalized keys in that interval, and adding add once for each selected element gives the actual sum. Thus every operation is correct.

Running time. Treap split, merge, insert, and erase take expected $O(\log q)$ time. The global add operation changes one variable and takes $O(1)$ time.

Problem 11. Maintaining the gcd of a dynamic set

Maintain a set of integers under insertion, deletion, and query for the gcd of all elements.

Solution.

Idea. The gcd is associative and insensitive to order:

$$\gcd(a, b, c) = \gcd(\gcd(a, b), c).$$

Thus it is a valid subtree aggregate in a balanced search tree.

Algorithm. Store the elements in a balanced BST. In each node v maintain

$$G(v) = \gcd(\text{key}(v), G(\text{left}(v)), G(\text{right}(v))),$$

where the gcd of an empty subtree is defined as 0. After every rotation, insertion, or deletion, recompute $G(v)$ for the affected nodes from their children.

The answer to a query is $G(\text{root})$; for the empty set we may define the answer to be 0.

Correctness. The invariant states that $G(v)$ is the gcd of all keys in the subtree of v . It is true for leaves. If it is true for the two children of v , then the displayed formula gives exactly the gcd of all values in v 's subtree. Since insertion, deletion, and rotations only change $O(\log n)$ nodes on the search/rebalancing path, recomputing this formula bottom-up restores the invariant. Therefore $G(\text{root})$ is the gcd of the whole set.

Running time. Each update changes $O(\log n)$ nodes in a balanced tree, and each recomputation uses a constant number of gcd operations. If all values are at most C , each gcd computation costs $O(\log C)$ bit operations, so updates cost $O(\log n \log C)$ bit operations. The query itself is $O(1)$.

Problem 12. Maintaining order statistics in a dynamic set

Maintain a set of numbers under insertion, deletion, and query for the k -th order statistic.

Solution.

Idea. Augment every node by the size of its subtree. Then the rank of the root separates the query into left subtree, root, or right subtree.

Algorithm. Use a balanced BST. In every node v store

$$\text{size}(v) = 1 + \text{size}(\text{left}(v)) + \text{size}(\text{right}(v)).$$

This value is updated after insertions, deletions, and rotations.

To find the k -th smallest element, use one-indexing and descend as follows:

```
node* kth(node* v, int k) {
    while (v != nullptr) {
        int leftSize = size(v->left);
        if (k <= leftSize) {
            v = v->left;
        } else if (k == leftSize + 1) {
            return v;
        } else {
            k -= leftSize + 1;
            v = v->right;
        }
    }
    return nullptr; // k was invalid
}
```

Correctness. For any node v , all keys in its left subtree are smaller than $\text{key}(v)$, and all keys in its right subtree are larger. Therefore the first $\text{size}(\text{left}(v))$ elements of the subtree order lie in the left subtree; the next element is v itself; all remaining elements lie in the right subtree. The descent follows exactly this trichotomy, adjusting k when the search moves into the right subtree. Hence it returns precisely the k -th smallest key.

Running time. Insertion and deletion cost $O(\log n)$ and maintain subtree sizes along the update path. The order-statistic query follows one root-to-leaf path, so it also costs $O(\log n)$.